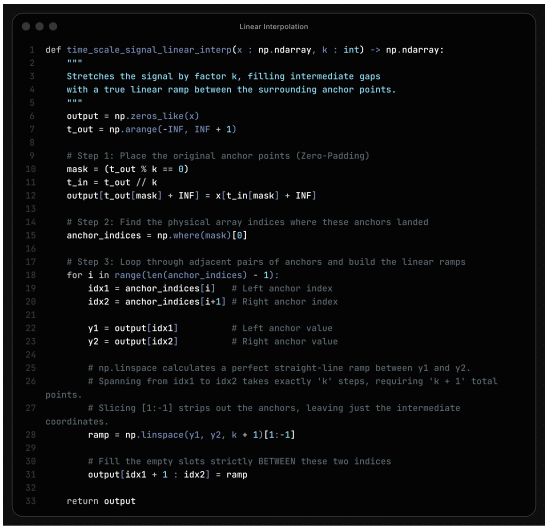
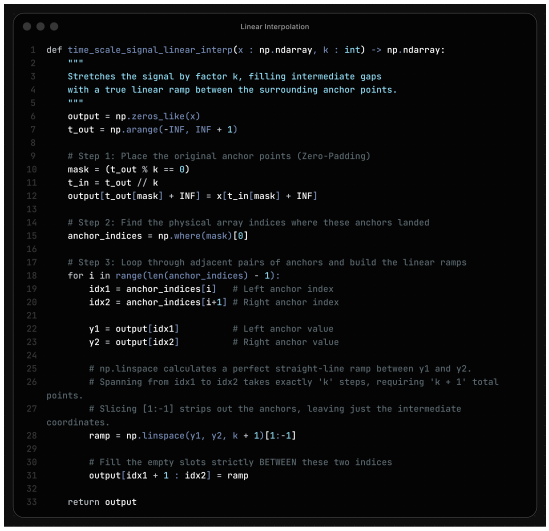
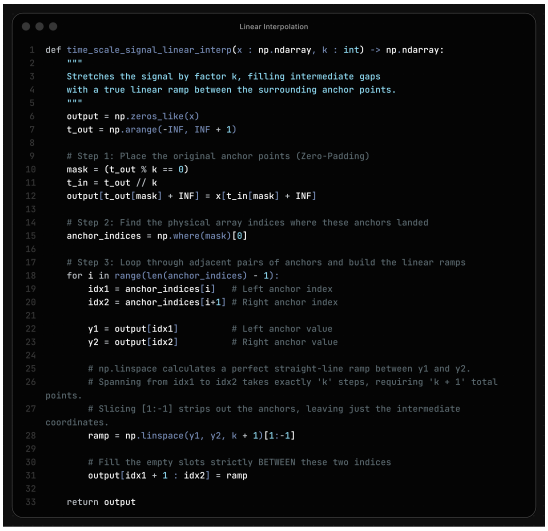


Figure 1: The reference waveform



(a) figure left



(b) figure right

Figure 2: whole figure

Ex 05

Lorem ipsum dolor sit amet, consectetuer adipiscing elit. Ut purus elit, vestibulum ut, placerat ac, adipiscing vitae, felis. Curabitur dictum gravida mauris. Nam arcu libero, nonummy eget, consectetuer id, vulputate a, magna. Donec vehicula augue eu neque. Pellentesque habitant morbi tristique senectus et netus et malesuada fames ac turpis egestas. Mauris ut leo. Cras viverra metus rhoncus sem. Nulla et lectus vestibulum urna fringilla ultrices. Phasellus eu tellus sit amet tortor gravida placerat. Integer sapien est, iaculis in, pretium quis, viverra ac, nunc. Praesent eget sem vel leo ultrices bibendum. Aenean faucibus. Morbi dolor nulla, malesuada eu, pulvinar at, mollis ac, nulla. Curabitur

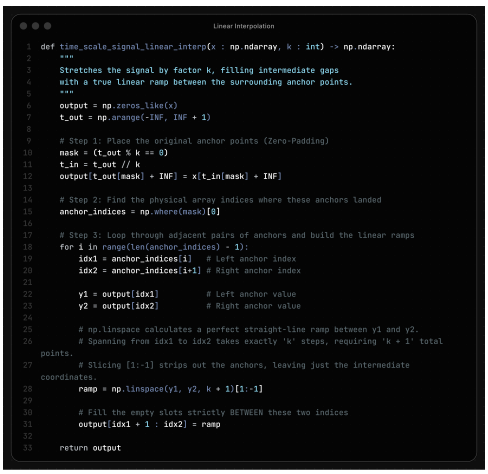


Figure 3: A wrapped figure n the right

auctor semper nulla. Donec varius orci eget risus. Duis nibh mi, congue eu, accumsan eleifend, sagittis quis, diam. Duis eget orci sit amet orci dignissim rutrum.

Nam dui ligula, fringilla a, euismod sodales, sollicitudin vel, wisi. Morbi auctor lorem non justo. Nam lacus libero, pretium at, lobortis vitae, ultricies et, tellus. Donec aliquet, tortor sed accumsan bibendum, erat ligula aliquet magna, vitae ornare odio metus a mi. Morbi ac orci et nisl hendrerit mollis. Suspendisse ut massa. Cras nec ante. Pellentesque a nulla. Cum sociis natoque penatibus et magnis dis parturient montes, nascetur ridiculus mus. Aliquam tincidunt urna. Nulla ullamcorper vestibulum turpis. Pellentesque cursus luctus mauris.

Figure 4 sits on the right while this paragraph flows around it. Lorem ipsum ... (enough text to fill the figures height)

Lorem ipsum dolor sit amet, consectetuer adipiscing elit. Ut purus elit, vestibulum ut, placerat ac, adipiscing vitae, felis. Curabitur dictum gravida mauris. Nam arcu libero, nonummy eget, consectetuer id, vulputate a, magna. Donec vehicula augue eu neque. Pellentesque habitant morbi tristique senectus et netus et malesuada fames ac turpis egestas. Mauris ut leo. Cras viverra metus rhoncus sem. Nulla et lectus vestibulum urna fringilla ultrices. Phasellus eu tellus sit amet tortor gravida placerat. Integer sapien est, iaculis in, pretium quis, viverra ac, nunc. Praesent eget sem vel leo ultrices bibendum. Aenean faucibus. Morbi dolor nulla, malesuada eu, pulvinar at, mollis ac, nulla. Curabitur auctor semper nulla. Donec varius orci eget risus. Duis nibh mi, congue eu, accumsan eleifend, sagittis quis, diam. Duis eget orci sit amet orci dignissim rutrum.

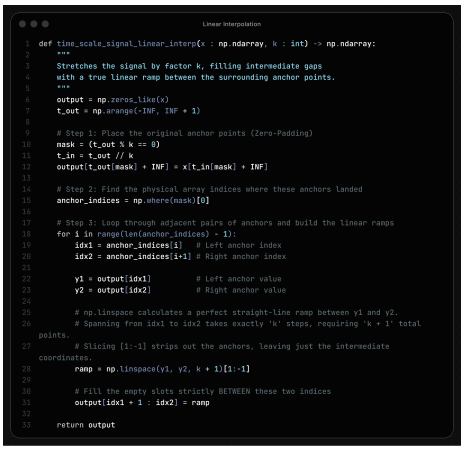


Figure 4: A wrapped figure on the right.

Nam dui ligula, fringilla a, euismod sodales, sollicitudin vel, wisi. Morbi auctor lorem non justo. Nam lacus libero, pretium at, lobortis vitae, ultricies et, tellus. Donec aliquet, tortor sed accumsan bibendum, erat ligula aliquet magna, vitae ornare odio metus a mi. Morbi ac orci et nisl hendrerit mollis. Suspendisse ut massa. Cras nec ante. Pellentesque a nulla. Cum sociis natoque penatibus et magnis dis parturient montes, nascetur ridiculus mus. Aliquam tincidunt urna. Nulla ullamcorper vestibulum turpis. Pellentesque cursus luctus mauris.

Ex 08

(a) Framed (b) Fixed height (c) Cropped (d) Stretched

Figure 5: Four ways to control the size and extent of an image.

1 Ex 09

Linear Interpolation

```python
def time_scale_signal_linear_interp(x : np.ndarray, k : int) -> np.ndarray:
    """
    Stretches the signal by factor k, filling intermediate gaps
    with a true linear ramp between the surrounding anchor points.
    """
    output = np.zeros_like(x)
    t_out = np.arange(-INF, INF + 1)

    # Step 1: Place the original anchor points (Zero-Padding)
    mask = (t_out % k == 0)
    t_in = t_out // k
    output[t_out[mask] + INF] = x[t_in[mask] + INF]

    # Step 2: Find the physical array indices where these anchors landed
    anchor_indices = np.where(mask)[0]

    # Step 3: Loop through adjacent pairs of anchors and build the linear ramps
    for i in range(len(anchor_indices) - 1):
        idx1 = anchor_indices[i]   # Left anchor index
        idx2 = anchor_indices[i+1] # Right anchor index

        y1 = output[idx1]          # Left anchor value
        y2 = output[idx2]          # Right anchor value

        # np.linspace calculates a perfect straight-line ramp between y1 and y2.
        # Spanning from idx1 to idx2 takes exactly 'k' steps, requiring 'k + 1' total points.
        # Slicing [1:-1] strips out the anchors, leaving just the intermediate coordinates.
        ramp = np.linspace(y1, y2, k + 1)[1:-1]

        # Fill the empty slots strictly BETWEEN these two indices
        output[idx1 + 1 : idx2] = ramp

    return output
```

Figure 6: Measured step response.

Table 1: Extracted parameters.

Quantity	**Value**
Time constant τ	10.0 ms
Rise time t_r	22.0 ms
Overshoot	4.3%